

GPU Collision Detection Report: Current Surgical Dense-Grid Path

Updated report for the current SOFA v25.12 + SofaGpuCollision implementation after removing the redundant object-level GPU AABB broad-phase work and moving primitive candidate generation into a GPU dense-grid path inside the narrow phase.

Workspace: C:\Users\arfin\Desktop\GPU SOFA

Environment: WSL2 wsl-gpu-proj

SOFA: v25.12

GPU: NVIDIA GeForce GTX 1650 Ti

Mode: detection benchmark, no response

Report date: 2026-05-07

1. Executive Summary

The broad phase has not been deleted from SOFA's pipeline, because `CollisionPipeline` still requires a broad-phase component to produce object-level collision model pairs. What changed is more important: the old GPU root-AABB culling kernel is disabled by default for the surgical two-object case.

For the target scene shape, one tissue object and one blade/tool object, the broad phase now acts as a small compatibility pass that emits the valid tissue/tool object pair. The actual broad filtering that matters for collision detection is now the GPU dense 3D grid inside `GpuCollisionNarrowPhase`.

Current Truth

- **Object-level GPU AABB culling:** disabled by default with `useObjectAabbCulling=false`.
- **SOFA broad phase:** still present, emits tissue/tool model pair.
- **Primitive candidate generation:** GPU dense grid.
- **Duplicate pair removal:** enabled with sort + unique.
- **Detection-only D2H:** counters only, 16 bytes per step in verified benchmark.
- **Remaining bottleneck:** CPU-staged triangle upload, 512480 bytes per step.

2. Full Current Workflow

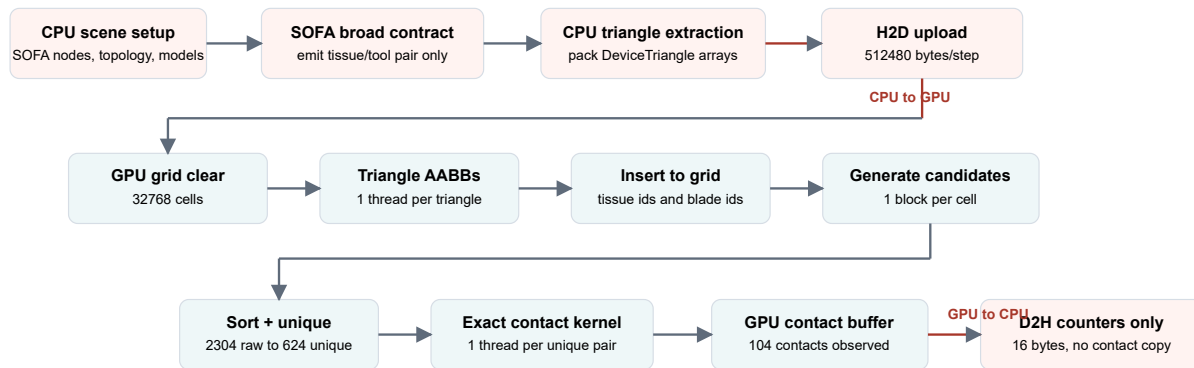


Figure 1. Current end-to-end workflow. The SOFA broad phase is now a compatibility pair emitter; dense-grid candidate generation, deduplication, and exact contacts happen on the GPU.

3. Broad Phase: What Was Removed and What Remains

The earlier report described a GPU root-AABB broad phase. That code still exists for general scenes, but it is no longer the default for the surgical two-object path. The current constructor default is:

```
d_useObjectAabbCulling = false
```

With that setting, `GpuCollisionBroadPhase` loops over the registered collision models, applies SOFA's intersector compatibility checks, and emits the valid model pairs. For one tissue and one blade this becomes exactly one object pair.

Broad-phase item	Current behavior	Why
Object-level GPU AABB culling	Disabled by default	One tissue and one blade means object-level culling adds latency but does not reject anything useful.
SOFA broad-phase component	Still required	<code>CollisionPipeline</code> expects a broad phase to feed the narrow phase.
Broad output	Usually one pair: tissue model vs blade model	The actual primitive filtering is now done in the dense grid.
Measured broad GPU kernel	0.000000 ms	No broad CUDA kernel is launched in the default surgical path.

Precise wording: broad phase is not totally removed from SOFA. The redundant object-level GPU broad-phase collision detection has been bypassed by default. The real broad filtering is now primitive-level dense-grid candidate generation inside the narrow phase.

4. Dense Grid Concept

The scene is divided into one shared 3D grid. There are not two separate grids. Each cell stores two lists: tissue triangle ids and tool triangle ids. If a tissue triangle and a tool triangle land in the same cell, they become a candidate pair.

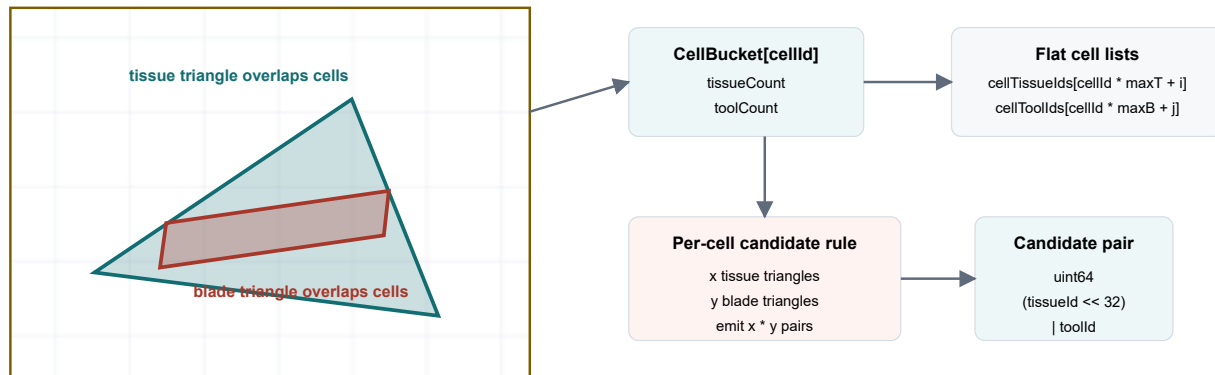


Figure 2. One global 3D grid. Every cell stores two triangle-id lists: tissue ids and tool ids. A triangle can overlap multiple cells, which is why duplicate candidate pairs must be removed.

4.1 Grid Data Structures

```
struct DeviceCellBucket
{
    uint32_t tissueCount;
    uint32_t toolCount;
};

DeviceCellBucket* grid;
uint32_t* cellTissueIds; // cellId * maxTissueTrianglesPerCell + localIndex
uint32_t* cellToolIds;   // cellId * maxToolTrianglesPerCell + localIndex
uint64_t* candidatePairs; // high 32 bits = tissue triangle id, low 32 bits = tool triangle id
```

4.2 Current Benchmark Grid Parameters

Parameter	Value
Grid min	[-4.5, -0.5, -4.5]
Grid max	[4.5, 0.5, 4.5]
Grid resolution	64 x 8 x 64 = 32768 cells
Contact distance	0.03
Max tissue ids per cell	128
Max tool ids per cell	64
Max candidate pairs	2000000

5. GPU Kernel Implementation Details

Order	Kernel / operation	Launch shape	Input	Output	Where data lives
1	clearDenseGridKernel	ceil(numCells / 256) blocks, 256 threads	grid buffer	zeroed tissueCount and toolCount	GPU global memory
2	triangleAabbKernel for tissue	1 thread per tissue triangle	DeviceTriangle[]	DeviceAabb[]	GPU global memory
3	triangleAabbKernel for blade/tool	1 thread per tool triangle	DeviceTriangle[]	DeviceAabb[]	GPU global memory
4	insertTissueTrianglesKernel	1 thread per tissue triangle	tissue AABBs, grid config	cell tissue id lists	GPU global memory with atomic counters
5	insertToolTrianglesKernel	1 thread per tool triangle	tool AABBs, grid config	cell tool id lists	GPU global memory with atomic counters
6	generateDenseGridCandidatePairsKernel	1 block per cell, 256 threads per block	cell tissue ids, cell tool ids	raw encoded candidate pairs	GPU global memory with atomic output counter
7	thrust::sort + thrust::unique	Thrust device operations	raw candidate pair array	unique candidate pair prefix	GPU memory
8	exactDenseGridContactKernel	1 thread per unique candidate pair	unique pairs, tissue triangles, tool triangles	DeviceExactContact[] and contact counter	GPU global memory

5.1 Candidate Generation Per Cell

```
// One CUDA block owns one cell.
cellId = blockIdx.x
x = min(grid[cellId].tissueCount, maxTissueTrianglesPerCell)
y = min(grid[cellId].toolCount, maxToolTrianglesPerCell)
totalPairs = x * y

for localPair = threadIdx.x; localPair < totalPairs; localPair += blockDim.x:
    tissueLocal = localPair / y
    toolLocal   = localPair % y
    tissueId = cellTissueIds[cellId * maxTissuePerCell + tissueLocal]
    toolId   = cellToolIds[cellId * maxToolPerCell + toolLocal]
    candidatePairs[atomicAdd(candidateCount, 1)] = encode(tissueId, toolId)
```

5.2 Duplicate Removal

Duplicate removal is necessary because a triangle AABB can overlap multiple grid cells. The same tissue/tool triangle pair can therefore be emitted by more than one cell. The current implementation deduplicates with:

```
thrust::sort(candidatePairs, candidatePairs + usedCandidateCount)
uniqueEnd = thrust::unique(candidatePairs, candidatePairs + usedCandidateCount)
exactCandidateCount = uniqueEnd - candidatePairs
```

In the verified one tissue / one blade benchmark:

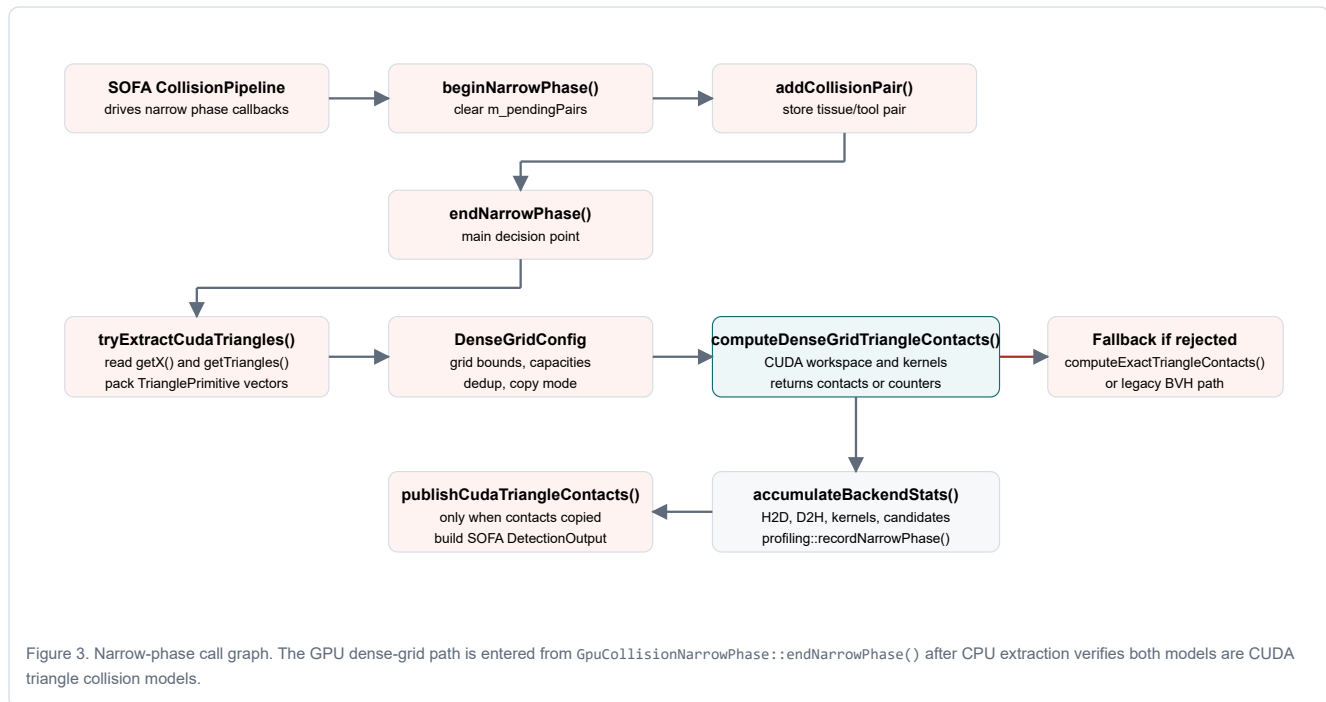
- raw candidate pairs: 2304
- unique candidate pairs after dedup: 624
- exact contacts emitted: 104

5.3 Exact Contact Kernel

The exact kernel runs one thread per unique candidate pair. Each thread decodes one `uint64_t` pair, loads one tissue triangle and one tool triangle, and runs the existing triangle-triangle Separating Axis Theorem test. Intersecting pairs write one compact `DeviceExactContact` record with an atomic contact counter.

6. Narrow-Phase Call Graph

The narrow phase is not just one CUDA kernel. It is a SOFA callback layer, a CPU staging layer, a persistent CUDA workspace layer, and then a sequence of GPU kernels/device operations. The current surgical path is selected when both collision models can be interpreted as `CudaTriangleCollisionModel` and `useDenseGrid=true`.



6.1 Exact Function Order in the Current Path

Order	Function / callback	CPU or GPU	What it does	Important data
1	<code>GpuCollisionNarrowPhase::beginNarrowPhase()</code>	CPU	Clears <code>m_pendingPairs</code> , then calls base SOFA narrow-phase begin.	empty pending pair vector
2	<code>GpuCollisionNarrowPhase::addCollisionPair(cmPair)</code>	CPU	Stores the tissue/tool pair instead of immediately handing it to SOFA BVH.	<code>std::vector<CollisionModelPair></code> <code>m_pendingPairs</code>
3	<code>GpuCollisionNarrowPhase::endNarrowPhase()</code>	CPU	Starts timing, iterates pending pairs, decides dense-grid GPU path vs fallback.	<code>profiling::StageSnapshot</code>
4	<code>tryExtractCudaTriangles(pair.first)</code> and <code>tryExtractCudaTriangles(pair.second)</code>	CPU staging	Dynamic-casts model to <code>CudaTriangleCollisionModel</code> , reads <code>getX()</code> and <code>getTriangles()</code> , packs <code>TrianglePrimitive</code> records.	<code>std::vector<TrianglePrimitive></code>
5	<code>Build backend::DenseGridConfig</code>	CPU	Copies SOFA component Data values into a plain backend config.	grid min/max/resolution, capacities, <code>deduplicatePairs</code> , <code>copyContactsToHost</code>
6	<code>backend::computeDenseGridTriangleContacts(...)</code>	CPU wrapper + GPU work	Runs allocation/reuse, H2D upload, dense-grid kernels, dedup, exact contact kernel, and D2H counters/contact data depending on mode.	<code>DenseGridWorkspace</code> , candidate pairs, contact buffer
7	<code>accumulateBackendStats(stageSnapshot, backendStats)</code>	CPU	Records GPU used, transfer bytes, kernel time, candidate/contact counts, allocation bytes.	<code>BackendExecutionStats</code> to <code>StageSnapshot</code>
8	<code>publishCudaTriangleContacts(...)</code>	CPU, only when contacts exist on host	Creates SOFA <code>DetectionOutput</code> records from copied contacts. Skipped in detection-only no-copy mode because contact vector is empty.	<code>TDetectionOutputVector<CudaTriangleModel, CudaTriangleModel></code>
9	<code>profiling::recordNarrowPhase(stageSnapshot)</code>	CPU	Publishes per-step profiling for the benchmark controller.	CSV and summary fields

7. Dense-Grid Backend Lifecycle

`computeDenseGridTriangleContacts()` is the main backend routine. It is a CPU function that orchestrates device memory, CUDA kernels, device synchronization, Thrust deduplication, and optional host contact publication.

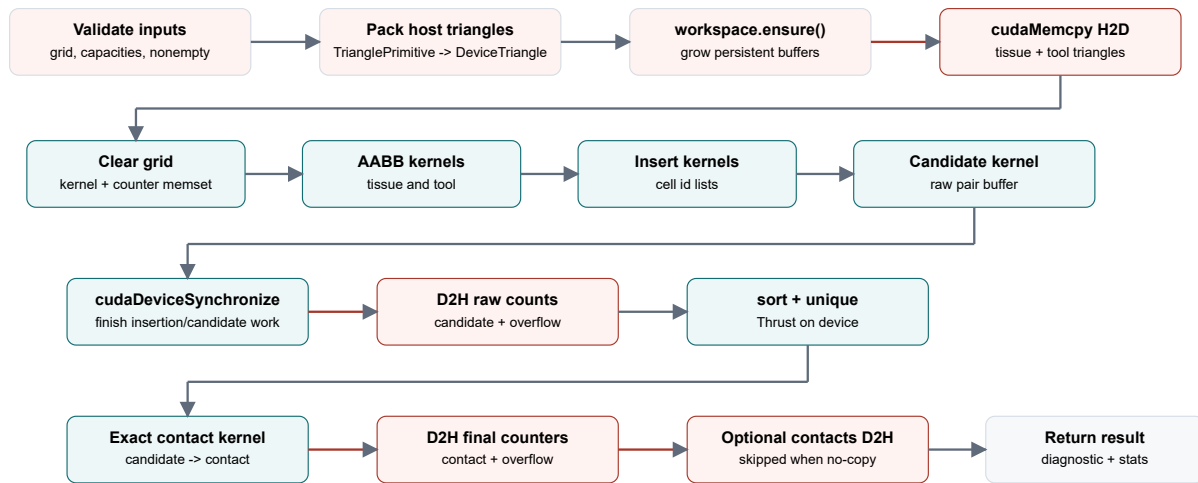


Figure 4. Backend lifecycle inside `computeDenseGridTriangleContacts()`. Red boxes/lines are CPU-side or transfer boundaries. Green boxes are GPU kernels or device operations.

7.1 Every CUDA Operation in the Dense-Grid Path

Order	Operation	Purpose	Transfer/accounting
1	workspace.ensure(...)	Allocates or reuses all dense-grid device buffers.	Counts new allocation bytes only when capacity grows.
2	cudaMemcpy(deviceTissueTriangles, hostTissueTriangles, tissueTriangleBytes, H2D)	Uploads packed tissue triangles.	Part of avg_host_to_device_bytes.
3	cudaMemcpy(deviceToolTriangles, hostToolTriangles, toolTriangleBytes, H2D)	Uploads packed blade/tool triangles.	Part of avg_host_to_device_bytes.
4	cudaEventCreate / cudaEventRecord(kernelStart)	Starts GPU kernel timing range.	No counted transfer.
5	clearDenseGridKernel<<<cellBlocks, 256>>>	Zeros per-cell tissue/tool counters.	GPU global memory writes.
6	cudaMemset(deviceCandidateCount, 0, 4)	Resets raw candidate counter.	Device-side memset.
7	cudaMemset(deviceContactCount, 0, 4)	Resets exact contact counter.	Device-side memset.
8	cudaMemset(deviceOverflowCount, 0, 4)	Resets overflow counter.	Device-side memset.
9	triangleAabbKernel for tissue	Computes inflated triangle AABBs.	GPU reads triangles, writes AABBs.
10	triangleAabbKernel for tool	Computes inflated blade/tool AABBs.	GPU reads triangles, writes AABBs.
11	insertTissueTrianglesKernel	Maps each tissue triangle AABB to all overlapped cells.	Atomic cell bucket writes.
12	insertToolTrianglesKernel	Maps each tool triangle AABB to all overlapped cells.	Atomic cell bucket writes.
13	generateDenseGridCandidatePairsKernel	For each cell, emits tissue x tool encoded pairs.	Atomic append to candidatePairs.
14	cudaDeviceSynchronize()	Waits before reading raw candidate and overflow counters.	Synchronization point.
15	cudaMemcpy(&rawCandidateCount, deviceCandidateCount, 4, D2H)	Copies raw candidate count.	Counter D2H.
16	cudaMemcpy(&overflowCount, deviceOverflowCount, 4, D2H)	Copies overflow count after insertion/candidate generation.	Counter D2H.
17	thrust::sort and thrust::unique	Deduplicates candidate pairs on the device.	GPU device operation.
18	cudaDeviceSynchronize()	Waits for Thrust dedup completion.	Synchronization point.
19	exactDenseGridContactKernel<<<contactBlocks, 256>>>	Runs exact triangle-triangle tests for unique candidates.	GPU contact buffer writes.
20	cudaDeviceSynchronize()	Waits for exact contact kernel.	Synchronization point.
21	cudaEventRecord(kernelEnd) / cudaEventElapsedTime	Records total dense-grid GPU time.	Benchmark metric.
22	cudaMemcpy(&hostContactCount, deviceContactCount, 4, D2H)	Copies exact contact count.	Counter D2H.
23	cudaMemcpy(&overflowCount, deviceOverflowCount, 4, D2H)	Copies final overflow count.	Counter D2H.
24	if (!copyContactsToHost) return true	Detection-only mode exits here, leaving contact records on GPU.	Final measured D2H is 16 bytes.

Order	Operation	Purpose	Transfer/accounting
25	cudaMemcpy(hostContacts, deviceContacts, hostContactCount * sizeof(DeviceExactContact), D2H)	Only in SOFA response compatibility mode.	Full contact-list copy.

8. Exact Contact Kernel Creation and Execution

The exact contact kernel is not launched over all tissue triangles times all tool triangles in the dense-grid path. It is launched only after candidate generation and deduplication. Therefore, exactCandidateCount determines the CUDA grid size.

```
const auto contactBlocks =
    static_cast<uint32_t>((exactCandidateCount + threadCount - 1) / threadCount);

exactDenseGridContactKernel<<<contactBlocks, 256>>>(<
    deviceTissueTriangles,
    deviceToolTriangles,
    deviceCandidatePairs,
    exactCandidateCount,
    deviceContacts,
    config.maxCandidatePairs,
    deviceContactCount,
    deviceOverflowCount);
```

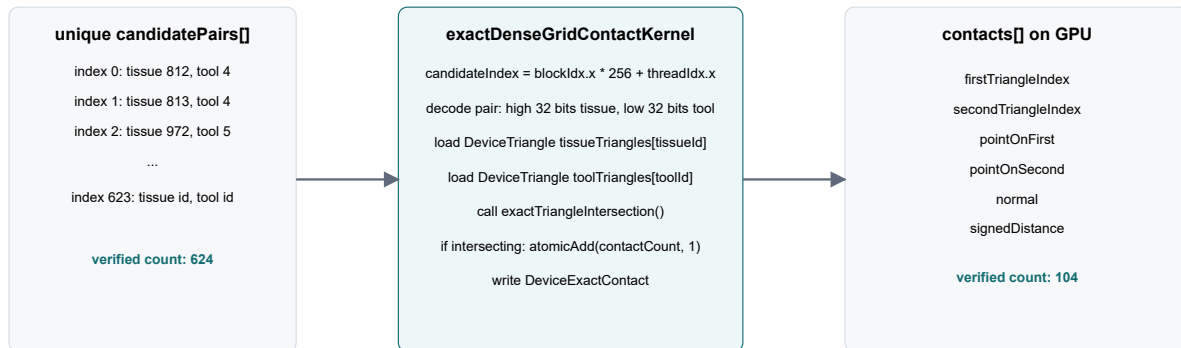


Figure 5. Exact contact kernel mapping. One CUDA thread owns one unique candidate pair. Only intersecting pairs append a DeviceExactContact using an atomic counter.

8.1 Exact Kernel Per-Thread Logic

```
candidateIndex = blockIdx.x * blockDim.x + threadIdx.x
if candidateIndex >= candidateCount:
    return

candidatePair = candidatePairs[candidateIndex]
tissueTriangleId = candidatePair >> 32
toolTriangleId = candidatePair & 0xffffffff

DeviceExactContact exactContact
if exactTriangleIntersection(tissueTriangles[tissueTriangleId],
    toolTriangles[toolTriangleId],
    exactContact):
    outputIndex = atomicAdd(contactCount, 1)
    if outputIndex < maxContactCount:
        contacts[outputIndex] = exactContact
    else:
        atomicAdd(overflowCount, 1)
```


9. Exact Triangle Intersection Details

exactTriangleIntersection() is the device function called by the exact contact kernel. It uses a triangle-triangle Separating Axis Theorem style test. The function returns false as soon as it finds a separating axis. If no separating axis is found, it builds a compact contact record.

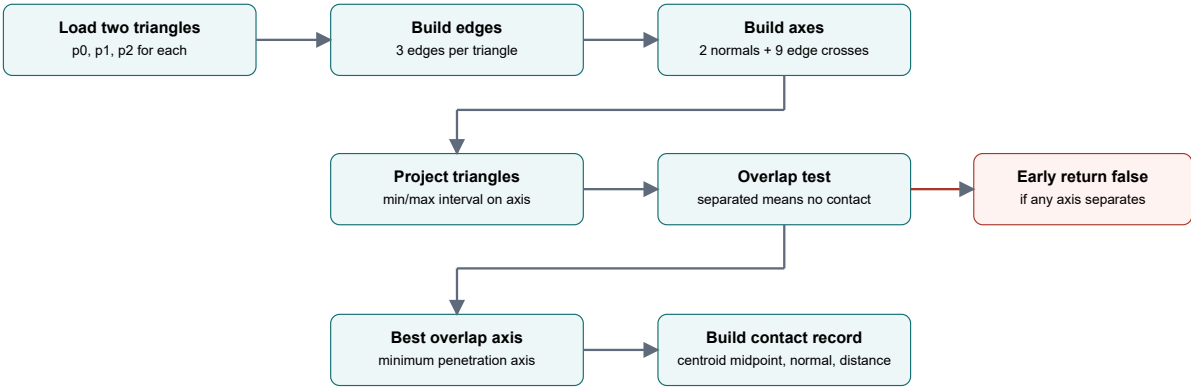


Figure 6. Exact triangle intersection flow. The function tests triangle normals and edge-cross axes. If all projections overlap, it creates a contact record from centroids and the best overlap axis.

9.1 Axes Tested

Axis source	Count	Code path	Purpose
First triangle normal	1	testSeparatingAxis(first, second, firstNormal, ...)	Separates triangles by plane normal of first triangle.
Second triangle normal	1	testSeparatingAxis(first, second, secondNormal, ...)	Separates triangles by plane normal of second triangle.
Edge cross products	9	cross3(firstEdges[i], secondEdges[j])	Separates by pairwise edge axes.

9.2 Contact Record Contents

```
outContact.firstTriangleIndex = first.triangleIndex
outContact.secondTriangleIndex = second.triangleIndex
outContact.pointOnFirst = midpoint(firstCentroid, secondCentroid)
outContact.pointOnSecond = midpoint(firstCentroid, secondCentroid)
outContact.normal = bestAxis, oriented from first to second
outContact.signedDistance = -bestOverlap
```

9.3 Important Accuracy Note

This exact contact implementation is suitable as a current GPU detection milestone and benchmark target, but it is not a full IPC-style robust contact solver. It uses a compact SAT-style overlap/contact approximation. For surgical-grade fast tools, the next realism steps are swept tests, edge-edge and point-triangle CCD, persistent manifolds, and then a GPU response solver.

10. CPU/GPU Transfer Map

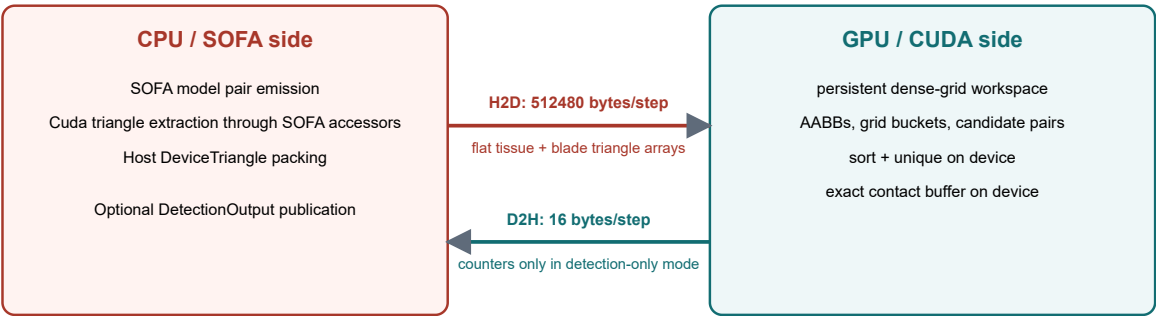


Figure 7. Current transfer boundary. The D2H side is now reduced to four counters for detection-only benchmarks. The remaining large transfer is H2D triangle staging.

Transfer	Direction	Payload	Verified value	Status
Broad phase	None for default surgical path	No root-AABB upload/download	avg_broad_kernel_ms = 0.000000	Removed as redundant latency
Narrow H2D	CPU to GPU	DeviceTriangle[] for tissue and blade	512480 bytes/step	Still the main bottleneck
Narrow D2H, detection-only	GPU to CPU	raw count, overflow count, contact count, overflow count recheck	16 bytes/step	Full contact list copy avoided
Narrow D2H, SOFA response mode	GPU to CPU	counter payload plus hostContactCount * sizeof(DeviceExactContact)	Disabled in verified benchmark	Required only when CPU SOFA response consumes contacts

11. Detection-Only Mode vs SOFA Response Mode

Detection-Only Benchmark Mode

copyContactsToHost = false

- GPU computes exact contacts.
- Full DeviceExactContact[] stays on GPU.
- CPU receives only counters for profiling and validation.
- No SOFA DetectionOutput is published.
- Correct for benchmarking detection and GPU residency.

SOFA Compatibility Response Mode

copyContactsToHost = true

- GPU computes exact contacts.
- Contacts copy back to CPU.
- GpuCollisionNarrowPhase publishes SOFA detection outputs.
- Required if using CPU/SOFA collision response.
- Less GPU-resident, but compatible with existing SOFA response.

Important: the verified benchmark disables collision response. SOFA prints a ContactManager warning because response is intentionally absent. That warning is expected for this detection-only report.

12. Verified Benchmark Result

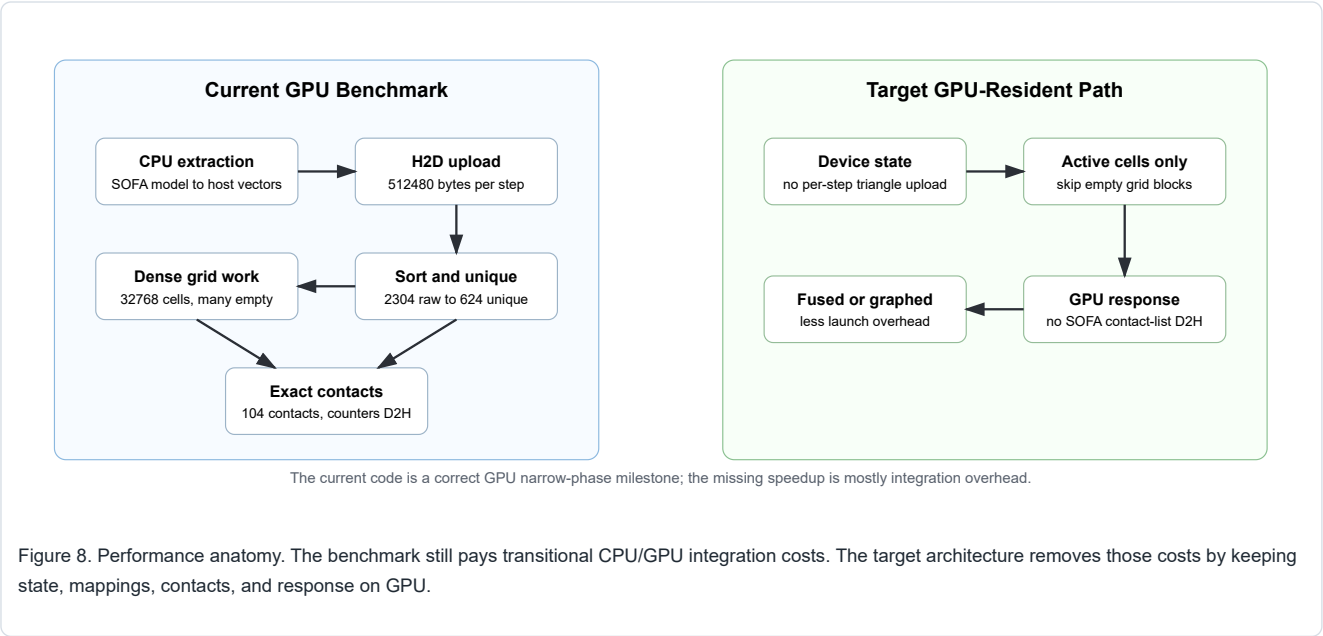
The final clean benchmark used unique labels so older CSV rows were not mixed into the result: `_verified_planb_20260507`. It ran 120 SOFA iterations with 10 warmup steps and 110 measured steps.

Metric	CPU scene	GPU dense-grid scene
Scene	<code>test_cpu_one_tissue_one_blade_benchmark.py</code>	<code>test_gpu_one_tissue_one_blade_dense_grid_benchmark.py</code>
Measured steps	110	110
Average step time	0.002071158 s	0.004351679 s
Average FPS	482.821613	229.796369
Average broad kernel time	not instrumented by plugin	0.000000000 ms
Average narrow kernel time	not instrumented by plugin	1.631790249 ms
Average H2D	not applicable	512480 bytes/step
Average D2H	not applicable	16 bytes/step
Average device allocation after warmup	not applicable	0 bytes/step
Candidate counts	not exported by CPU scene	2304 raw, 624 unique, 104 contacts
Overflow count	not applicable	0

12.1 Why CPU Is Still Faster Here

The implementation is good for the direction we want, but this benchmark does not yet measure the final GPU-resident architecture. It measures a transitional integration path: SOFA still owns the collision model at the CPU boundary, the plugin packs triangle records on the host, uploads all triangle records every step, launches several small kernels and device operations, synchronizes to read counters, and then returns to SOFA. For a small one-blade workload, that fixed overhead is larger than the actual triangle-triangle work.

The CPU path is not doing something conceptually better. It is simply mature, local, and cheap for this size: the data is already in host memory, `ParallelBVHNarrowPhase` is optimized, and the scene has only one tissue surface and one blade. The GPU path will start winning when the expensive parts stay resident on device and when the benchmark has enough contact work to amortize launch, synchronization, deduplication, and upload overhead.



12.2 Bottleneck Diagnosis

Bottleneck	Current evidence	Why it hurts this benchmark	Fix required for speedup
Per-step H2D triangle upload	512480 bytes/step	The benchmark uploads the complete tissue and blade triangle records every step before any GPU work starts.	Keep positions, triangle topology, AABBs, and mapped collision geometry resident on GPU.
Dense grid launches over all cells	$64 * 8 * 64 = 32768$ cells	One block per cell is simple and parallel, but most cells are empty for one blade touching a local patch.	Compact active cells, or switch candidate generation to a sorted sparse hash list.
Duplicate removal cost	2304 raw pairs become 624 unique pairs	Sorting a few thousand pairs can cost more than simply testing them on CPU for this tiny case.	Use active-cell compaction plus canonical owner-cell rules, or keep dedup only for larger/contact-response runs.
Kernel launch and synchronization overhead	clear, AABB, insert, pair generation, sort/unique, exact contacts, counter copies	The useful exact work is small, so fixed GPU orchestration overhead dominates the step time.	Fuse kernels where practical and capture the stable step with CUDA graphs.
Only detection is GPU-resident	copyContactsToHost=false leaves records on GPU, but SOFA still drives the frame	The benchmark proves low D2H for detection, not full GPU simulation throughput.	Move FEM state, mappings, solver, contact metadata, and final response onto the GPU path.
CPU baseline is strong for small scenes	ParallelBVHNarrowPhase with host-resident data	For one blade and one modest tissue surface, CPU cache locality and mature SOFA code are hard to beat.	Benchmark larger/deeper contact scenes after removing residency and launch overhead.

So the conclusion is not that the GPU implementation is bad. The conclusion is sharper: the current dense-grid narrow phase is architecturally useful and correctness-friendly, but the benchmark is too small and too CPU-integrated to expose a speedup. The next speed milestone is not another triangle test; it is eliminating the per-step upload, avoiding empty-cell work, reducing synchronization, and benchmarking after the collision geometry is genuinely device-owned.

13. Memory Residency and Allocation Behavior

The dense-grid backend uses a persistent DenseGridWorkspace. It grows buffers on demand and then reuses them across frames. In the verified benchmark, the first GPU row allocated 138247948 bytes because the workspace had to be created for the larger per-cell capacities. After that, allocation was 0 bytes per step, and the measured post-warmup average was 0.

Buffer	Role	Lifetime
tissueTriangles, toolTriangles	hot triangle inputs	persistent, overwritten each step
tissueAabbs, toolAabbs	per-triangle inflated AABBs	persistent, recomputed each step
grid	per-cell tissue/tool counters	persistent, cleared each step
cellTissueIds, cellToolIds	flat bucket storage	persistent, rewritten each step
candidatePairs	raw and unique encoded pairs	persistent, sorted in place
contacts	exact contact records	persistent, remains device-side in detection-only mode
candidateCount, contactCount, overflowCount	device counters	persistent, reset each step

14. Mapping and Data Ownership

In the benchmark scenes, the collision surfaces are direct CudaVec3f triangle surfaces, so there is no separate render mesh or simulation-to-collision mapping. In the broader surgical architecture, the intended mapping is still:

```
simulation tet mesh on GPU
-> GPU collision surface / collision tet mesh
-> GPU render mesh
-> GPU dense-grid detection
-> GPU response later
```

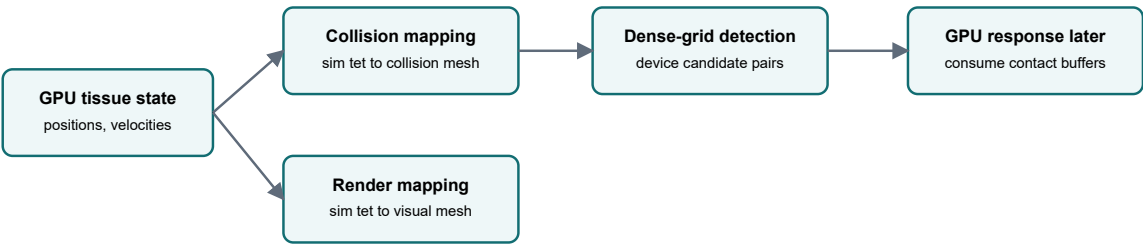


Figure 9. Intended GPU-resident mapping architecture. The current benchmark uses direct collision surfaces; future tissue simulation should keep mapping outputs on the GPU before dense-grid detection.

15. Fallback and Correctness Guards

- If dense-grid capacity overflows, the narrow phase rejects the grid result and can fall back for correctness.
- If models are unsupported or not CUDA triangle models, the legacy SOFA/BVH path remains available.
- allowCPUFallback=true keeps the scene from failing hard during unsupported combinations.
- Overflow count is exported and was verified as 0 in the final benchmark.
- Dedup is enabled by default with deduplicatePairs=true.

16. Verification Checklist

Check	Result
Plugin build and install in WSL	Passed. Built and installed libSofaGpuCollision.so.
Scene syntax check	Passed. python -m py_compile passed for the new benchmark scenes.
Final CPU benchmark	Passed. 120 iterations, 110 measured.
Final GPU benchmark	Passed. 120 iterations, 110 measured.
Dense-grid fallback warning in final run	None observed. The earlier capacity overflow was fixed by increasing per-cell and candidate capacities.
Broad-phase kernel removed from default path	Verified. avg_broad_kernel_ms = 0.00000000.
Duplicate removal	Verified. 2304 raw pairs reduced to 624 unique pairs.
No contact-list D2H in detection-only mode	Verified. avg_device_to_host_bytes = 16.
Persistent allocation after warmup	Verified. avg_device_allocation_bytes = 0.

17. Final Takeaways

- The redundant object-level GPU AABB broad phase is no longer active for the surgical two-object path.
- The SOFA broad phase remains only as a contract to emit the tissue/tool object pair.
- The dense 3D grid inside narrow phase is now the real primitive-level broad filter.
- Candidate duplicate removal is implemented and verified.
- Detection-only GPU mode avoids full contact D2H transfer; only counters copy back.
- The remaining major transfer is CPU-staged triangle H2D upload.
- For small one-tool scenes, CPU can still be faster because the GPU path has staging, kernel launch, sort, and synchronization overhead.
- The next performance milestone is direct device-buffer integration for SOFA CUDA collision geometry and later GPU response consumption of the contact buffer.

Source cross-checks used for this report: GpuCollisionBroadPhase.cpp, GpuCollisionNarrowPhase.cpp, GpuCollisionBackend.cu, test_gpu_one_tissue_one_blade_dense_grid_benchmark.py, and verified benchmark summaries under /opt/sofa/benchmark_logs.